



중간고사 대비

Orientation

- Computer Graphics
 - Modeling
 - 표현하고자 하는 물체를 Creating or Capturing
 - Rendering
 - Display device에서 보이게할 이미지 생성
 - Animating
 - 시간에 따라 오브젝트가 움직이게 하는 것

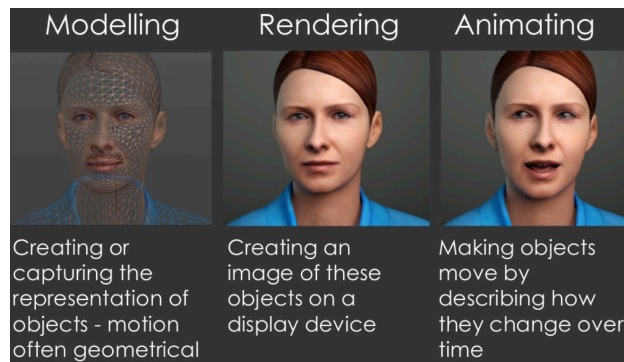
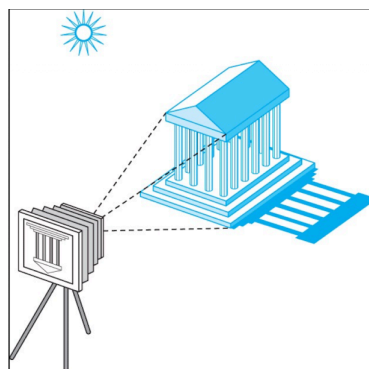


Image Formation and OpenGL

Image Formation (이미지 형성)

Elements of Image Formation (요소 4가지)

- 이미지 형성에 필요한 것
 - 모양 (Object)
 - 색상 (Materials)
 - 카메라의 위치 (Viewer)
 - 조명 (Light Sources)



- 더 추가적으로 필요한 건?
 - 그리는 스타일 (Attributes)
 - : Govern how light interacts with the materials in the scene.



Attributes is getting more important

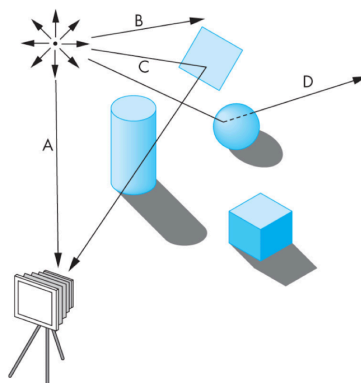
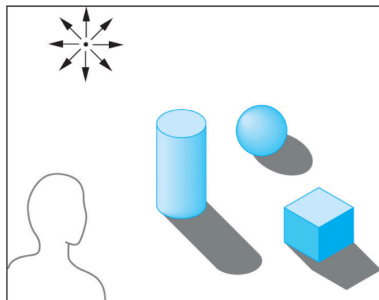
Separation is Important (분리하기)

Objects, Viewer, Light source and Attributes를 분리하는 것이 중요하다.

- Leads to simple software API.
 - Specify objects, lights, camera, attributes
 - Let implementation determine image
- Leads to fast hardware implementation.

Determine the color (색 결정하기)

- One way to form an image is to follow rays of light from the lights.
 - : 이미지를 형성하는 한 가지 방법은 빛에서 나오는 광선을 따라가는 것
- Each ray of light may have multiple interactions with objects.
 - : 각 광선은 물체들과 다양한 상호작용을 가지고 있음



Ray Tracing

Follow rays of light from center of projection until they either are absorbed by objects or go off to infinity.

→ 투영의 중심에서부터 물체에 흡수되거나 무한대로 발산할 때까지 광선을 따라간다.

광선의 발사 위치는 카메라(또는 눈)에서 발사한다. (Scene을 관통한다.)

→ 물체에 닿을 때까지 추적(혹은 무한대로 발산)

→ 닿은 지점에서 반사(Reflection), 굴절(Refraction), 그림자(Shadow) 등을 계산



결론적으로, 이 과정을 통해 "광선(Ray)"이 어떻게 색을 결정하는가를 역추적하는 방식이다.

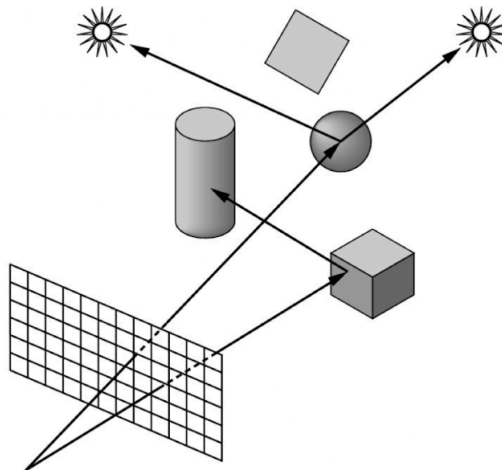
- **Can handle global effects (당연히 전역적인 효과를 볼 수 있음)**

- Multiple reflections (다중 반사)
- Translucent objects (반투명 객체)

- **Slow (계산 시간 개 오래걸림 → 당연함)**

- **Must have whole data base available at all times**

→ 당연히 전체 데이터가 램에 올라와야 함 (그래야 전역 계산 가능) → 사실상 불가능



→ 전통적인 CPU 렌더링 방식을 이용할 때 어려운 것



- 빛이 2회 이상 튕겨 반사광이 보이면 Global이라고 한다.
- 그것이 아니고 직선으로 바로 들어온다면 Local이라고 한다.

Why not ray tracing?

그래서 왜 안쓸까?

- Ray tracing seems more physically based so why don't we use it to design a graphics system?
: 레이 트레이싱은 물리적 기반인 것 같으니 그래픽 시스템을 설계하는 데 사용해 보는 건 어떨까요?
- Possible and is actually simple for simple objects such as polygons and quadrics with simple point sources

: 폴리곤이나 쿼드릭과 같은 간단한 객체에 대해 가능하며 실제로 간단합니다.

- In principle, can produce global lighting effects such as shadows and multiple reflections but ray tracing is slow and not well-suited for interactive applications

: 원칙적으로 그림자와 다중 반사와 같은 전역 조명 효과를 생성할 수 있지만 광선 추적은 느리고 인터랙티브 애플리케이션에 적합하지 않습니다.

- Now, Ray tracing with GPUs is real time

: 이제 GPU를 사용한 레이 트레이싱이 실시간으로 가능

The Journey of RTX | Powering Full Ray Tracing In Cyberpunk 2077

Explore the different RTX technologies that work together to make full ray tracing, also known as path tracing, possible in the beautiful, expansive, and immersive world of Cyberpunk 2077.

<https://www.youtube.com/watch?v=gq1klB4inoc>



물론 GPU 시대로 오면서 훨씬 빠른 Ray Tracing이 가능해졌지만, 완벽한 "Full Path Tracing"은 매우 무겁다.

1. 픽셀마다 수십 ~ 수백 개의 광선이 재귀적으로 발생
2. 각 광선마다 수많은 삼각형과 교차 검사 필요
3. 간접 조명(Bounce Lighting)까지 계산하면 연산량 폭증
4. CPU의 VRAM 한계 (모든 장면 데이터를 실시간으로 보관)

⇒ 그래서 RTX on 게임들도 "Hybrid Rendering" 방식 사용

Rasterization(OpenGL/DirectX 전통 방식) + Ray Tracing 일부 단계만 적용

E.g., 반사, 그림자, AO(Ambient Occlusion)만 Ray Tracing

→ 즉, GPU가 실시간으로 처리할 수는 있지만, 연산량이 워낙 커서 모든 조명효과를 완전히 Ray Tracing으로 대체하지는 못함

"Simplified" Approach

- Process objects one at a time in the order they are generated by the application
 - Can consider only local lighting

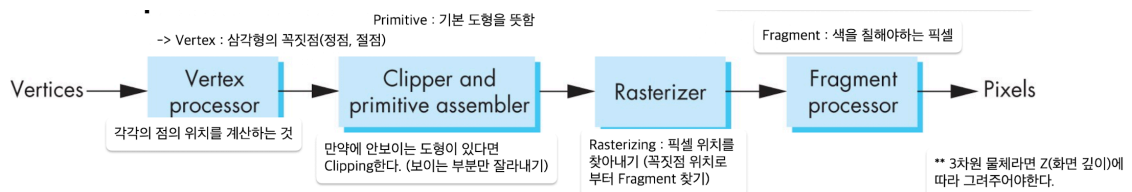
→ 일단 3차원 도형을 그리고 보는 것으로 통쳐보자.

→ 모든 삼각형에 대한 정보를 Streaming한다. (from CPU to GPU)

- Pipeline Architecture - Fixed Rendering Pipeline임

장점 : 하드웨어로 미리 만들 수 있음 (매우 빠름)

→ CPU를 통해 값만 넣어주면 됨



1. [Vertices] - 삼각형의 꼭짓점(정점, 절점)

2. **Vertex Processor**

: 각각의 점의 위치를 계산하는 과정

3. **Clipper and primitive assembler**
: 만약에 안 보이는 도형이 있다면 Clipping한다. (보이지 않는 부분 제거)

4. **Rasterizer**
: 픽셀의 위치를 찾아내기 (꼭짓점 위치로 Fragment 생성)

5. **Fragment processor**
: 픽셀에 칠할 색 결정

6. [Pixels] → Frame Buffer

- Screen Buffer

: 색상을 저장하는 버퍼

- Depth-Buffer

: 카메라로부터의 거리를 저장하는 버퍼

→ 이를 활용하여 거리 순으로 그림을 그린다면(덧칠하는 식), Sorting의 과정이 생략되어 훨씬 빠른 그리기 가능

- All steps can be implemented in hardware on the graphics card.

: 모든 작업은 GPU에서 이루어짐.

Vertex Processing : 점의 위치를 결정하는 과정

- Much of the work in the pipeline is in converting object representations from one coordinate system to another

: Pipeline에서의 대부분의 작업은 Object를 표현하는 것에서 좌표계를 변환하는 것입니다.

- Object coordinates
- Camera (eye) coordinates
- Screen coordinates

→ 위 세 단계를 거치며 Projection



좌표 체계는 원점이 존재해야 합니다. (기준이 있어야 좌표체계가 구성됨)

Object coordinates → Camera(eye) coordinates → Screen coordinates

- Every change of coordinates is equivalent to a matrix transformation

: 모든 좌표 체계 변환은 행렬 변환으로 작동함.

Rasterization

- If an object is visible, the appropriate pixels in the frame buffer must be assigned colors.

: 보이는 객체라면, Frame buffer 안의 적절한 픽셀에 색상이 할당되어야 함.

- Rasterizer produces a set of fragments for each objects

: Rasterizer는 각각의 Object들의 Fragment의 집합을 생성

- **Fragments are "Potential pixels"**

: Fragment는 잠재적 픽셀임

- Have a location in frame buffer

: Frame buffer안에 존재하고

- **Color and depth attributes**

: 색상과 Depth 버퍼를 가짐

- Vertex attributes are interpolated over objects by the rasterizer.
: Vertex attributes는 rasterizer에 의해 물체 위로 보간됨.

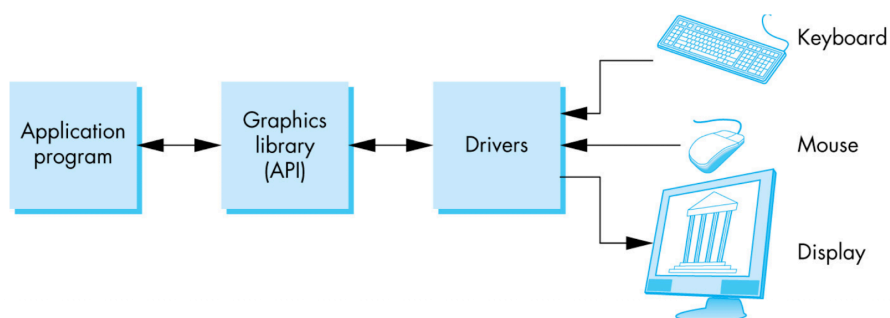
Fragment Processing

- Fragments are processed to determine the color of the corresponding pixel in the frame buffer.
: Frame Buffer에서 Fragments는 상응하는 픽셀의 색상 값을 결정하도록 처리됨.
- **Colors can be determined by texture mapping or interpolation of vertex colors.**
: 색상들은 Texture mapping이나 Vertex 색상의 보간으로 결정될 수 있음
→ 외부의 이미지(Texture)를 가져와서 칠하기 가능하다.
- Fragments may be blocked by other fragments closer to the camera.
: Fragments는 카메라에 더 가까운 다른 Fragments들에 의해 Block될 수 있음
 - Hidden-surface removal
: 숨겨진 표면을 지운다.

OpenGL : the API

The Programmer's Interface

- Programmer sees the graphics system through a software interface : the Application Programmer Interface (API)



API Contents

- Functions that specify what we need to form an image
 - Objects
 - Viewer
 - Light Sources
 - Attributes (Materials)
- Other information
 - Input from devices such as mouse and keyboard
 - Capabilities of system

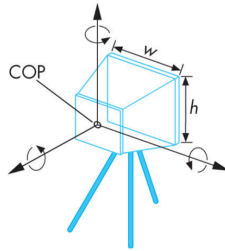
Objects Specification

- Most APIs support a limited set of primitives including

- Points (0D object)
 - Line segments (1D object)
 - Polygons (2D objects)
 - Some curves and surfaces
 - Quadratics
 - Parametric polynomials
 - All are defined through locations in space or vertices.
- : 모두 공간이나 Vertices의 위치를 통해 정의됨

[Old Version]

```
glBegin(GL_POLYGON) // Type of Object
glVertex3f(0,0, 0,0, 0,0); // Location of Vertex
glVertex3f(0,0, 0,0, 0,0);
glVertex3f(0,0, 0,0, 0,0);
glEnd(); // End of Object Definition
```



- Six degrees of freedom
 - Position of center of lens
 - Orientation
- Lens
- Film size
- Orientation of film plane

→ `glPerspective(...)` , `gluLookAt(...)`

Light and Materials(Old Style)

- Types of lights
 - Point sources vs. distributed sources
 - Spot lights
 - Near and far sources (근 및 원거리)
 - Color properties
- Material properties
 - Absorption(흡수) : color properties
 - Scattering (산란)
 - Diffuse (확산)
 - Specular (정반사)

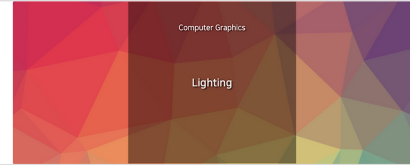
→ `glLight()` , `glMaterial()`

Lighting

Reflection of Light Light 빛은 물체에 의해 다음과 같이 될 수 있다. absorbed : 흡수 emitted : 발산 scattered : 산란 reflected : 반사 refracted : 굴절 그 중 scattering(산란)과 reflection(반사)이 물체의 표면의 시각적인 모습을 결정하는 결정적인 요인이다. 물체의 색상, highlight 등을 결정한다. scattering과 reflection을 합쳐서 reflection이라고



<https://gofo-coding.tistory.com/entry/Lighting>



Programming with OpenGL in a modern way

- Performance is achieved by using GPU rather than CPU
- **Control GPU through programs called shaders**

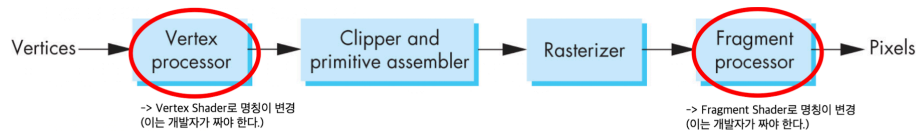
[Fixe Rendering Pipeline]

- 위에서는 장점만 이야기했지만, 단점도 존재한다.
- 하드웨어를 통해 연산이 수행되기에, 할 수 있는 것만 가능하다는 단점이 존재.

→ 이 단점을 극복하기 위해 원래는 정해진 값을 받았지만, 함수를 넣을 것이다. (SW의 추가)

옛날에는 정해진 방법을 빠르게 하기 위해, 그래픽 가속기를 통해 삼각형을 빠르게 처리하는 것만을 중요하게 했음. 그러나 현대의 방법은 정해진 값만 주는 것이 아닌, 함수의 형태로 개발자가 직접 Shaders를 짜서 결과값을 만들어 낸다.

→ 즉, **Modern graphics Pipeline**의 핵심은 **Vertex Processor, Fragment Processor**를 소스코드로 돌리는 것이다.

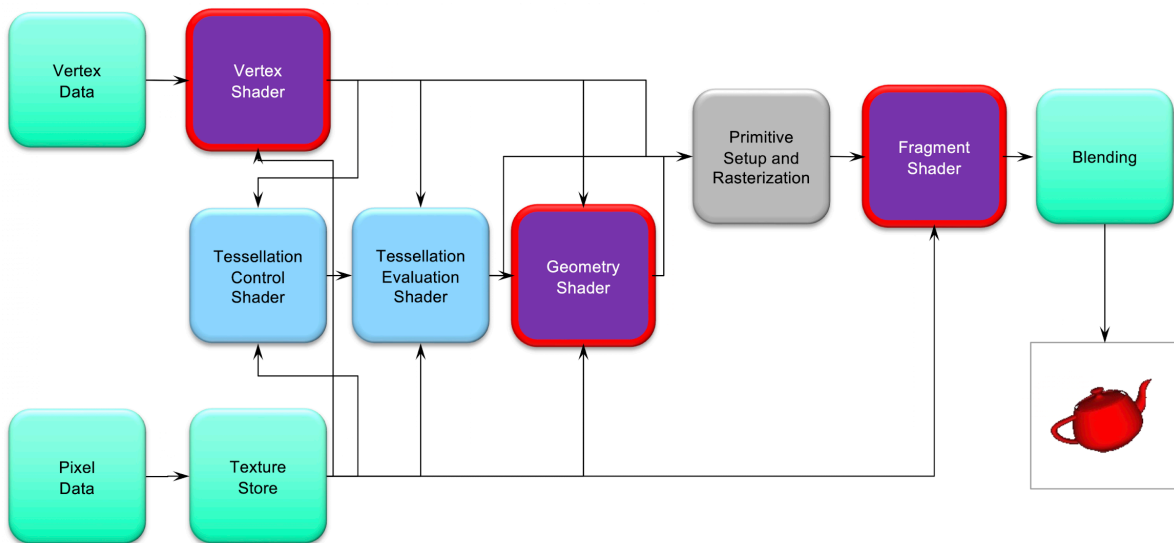


"new" OpenGL

- We'll concentrate on the latest versions of OpenGL
 - The currently available version is 4.5
 - At least higher than OpenGL 3.0 will work fine with the class
- They enforce a new way to program with OpenGL
 - **Allows more efficient use of GPU resources**
: 더 효율적으로 GPU 자원을 사용할 수 있게 해줌
- **modern OpenGL doesn't support**
: 최신 OpenGL은 다음을 지원하지 않는다.
 - Fixed-function graphics operations
 - lighting
 - transformations
- **All applications must use shaders for their graphics processing.**
: 반드시 Shader를 사용해야 함.

The Latest Pipelines

- Latest version is 4.5 (2014 기준)



- Texture store
: 전역적인 정보를 담는 이미지

여러 숫자를 던지고, 코드적으로 삼각형을 쪼개게 함. 예를 들어, 구를 만든다고 했을 때 부드럽게 만드려면 수많은 삼각형이 필요하게 됨. 그러나, 실제로 구를 만드려면 데이터는 4개 정도면 충분함. (중심, 반지름) 이를 효율적으로 만들기 위해 알고리즘을 구현하는 것이다. (즉, 4개의 데이터를 가지고 구를 만드는 것)

또한 삼각형의 데이터를 GPU의 메모리로 가져오는 것은 너무 느릴 수 있다(CPU→GPU 병목). 따라서 가장 작은 크기의 데이터로 만들고, 이를 GPU에서 직접 삼각형을 연산하게 하는 것이 가장 빠른 방법이 될 것이다. 이를 Geometry Shader에서 수행한다.

OpenGL Libraries

- OpenGL Core Library → Cross platform
 - OpenGL32 on Windows
 - GL on most Unix / Linux Systems (libGL.a)
- OpenGL Utility Library → 그림과는 상관없지만, 카메라의 위치 등의 추상적인 것들을 관리
 - Provides functionality in OpenGL core but avoids having to rewrite code
 - Will only work with legacy code
- Links with window system → GLUT로 통합
 - GLX for X window systems
 - WGL for Windows
 - AGL for Macintosh

GLUT

- OpenGL Utility Toolkit(GLUT) → 운영체제 소통용
 - Provides functionally common to all window systems
 - Open a window
 - Get input from mouse and keyboard
 - Menus (메뉴)

- **Event-Driven (이벤트 중심)**

- Code is portable but GLUT lacks the functionality if a good toolkit for a specific platform
 - No slide bars

→ 좋긴한데, 기능이 부족함

freeGLUT

- GLUT was created long ago and has been unchanged
 - Amazing that it works with OpenGL 3.1
 - Some functionality can't work since it requires deprecated functions
: 일부 기능은 더 이상 사용되지 않는 기능을 필요로 함. 그래서 작동할 수 없음.
 - freeGLUT updates GLUT → 오픈소스 기반
 - Added Capabilities (추가 기능)
 - Context Checking
- freeGLUT을 통해 GLUT를 업데이트 함

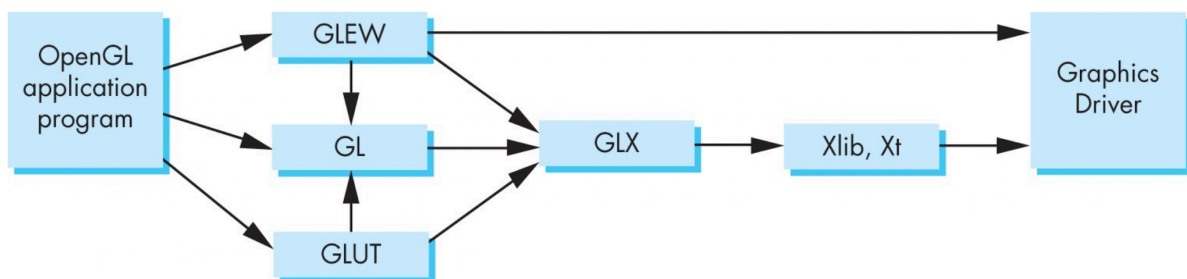
GLEW

- OpenGL Extension Wrangler Library

→ Wrangler : C언어이기에 Function Overloading이 안됨. 그래서 버전과 회사명을 세팅하면 자동으로 그 함수를 찾아주는 역할을 함

- Makes it easy to access OpenGL extensions available on a particular system
- Avoids having to have specific entry points in Windows code
- Applications needs only to include `glew.h` and run a `glewInit()`

Software Organization



Three different drawing approaches : Many Points

| Random Dots을 찍는 예제로 3가지 다른 Drawing 접근법을 확인하자.

Three Approaches

- Immediate mode graphics
 - 그릴 때마다 연산

- Retained mode graphics
: 데이터 미리 연산해두기
- GPU based graphics
: GPU로 연산하기

Immediate mode graphics

- Generate one primitive at a time and draw it immediately

```
void display()
{
    // 데이터를 만들고 바로 그리고의 반복
    for(num_points)
    {
        p = generate_a_point();
        Draw_a_point(p);
    }
}
```

→ 데이터를 만들고 바로 그리고의 반복



[중요]

- CG에서 Primitives는 보통 점, 선, 삼각형, 사각형, 다각형과 같이 그래픽을 구성하는 요소들을 말함

Retained mode Graphics

- Generate all primitives first, then draw them all

```
// 데이터가 고정되어 있다면, 먼저 데이터들을 만들어 CPU에 저장
p[num_points];
void initialize()
{
    for(num_points)
    {
        // 데이터를 만들고, 저장한다.
        q = generate_a_point;
        p = store_the_point(q);
    }
}

void display()
{
    draw_all_the_points(p); // GPU에 보내서 그리기
}
```

→ 데이터들을 모두 만들고 저장 후 GPU에 보냄

→ CPU가 모든 데이터를 처리하고, 저장하고 있음. GPU는 단순히 받은 데이터를 렌더링만 함.

→ CPU가 draw 명령 수행 (GPU는 그리기만 함)



근데, 이를 확실하게 하려면 CPU의 데이터를 벌크로 보냈을 때 GPU가 알아먹을 수 있어야 함. 또한 GPU의 메모리가 충분히 커야 함.

GPU-based Graphics

- Generate All the points first, send them to GPU and then draw them.

```
p[num_points];
void initialize()
{
    for(num_points)
    {
        q = generate_a_point();
        p = store_the_point(q);
    }
    // GPU에 데이터를 보내고, 어떻게 그릴지 명령해야 함
    send_all_points_to_GPU(p);
}

void display()
{
    display_data_on_GPU();
}
```

→ CPU가 명령만 보내고, 실제 연산 및 그림은 GPU가 처리

→ CPU는 단순히 draw 명령을 GPU에 전달함.



Retained mode vs. GPU-based

- 연산 처리의 차이점
 - Retained는 CPU에서 연산 후 GPU는 단순히 그림 (단순 그림 도구)
 - GPU-based는 GPU에 데이터를 넘겨주면 연산 처리 후 그림 (데이터 + 연산 + 그림 처리)

A First Program : Many Points with GPU

초기 세팅

```
int main(int argc, char ** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE, GLUT_RGBA);
    glutInitWindowSize(512, 512);
    glutCreateWindow("Many Points GPU");

    // For Using the Modern OpenGL (최신 OpenGL로 실행)
    glewExperimental = true;
    glewInit();
```

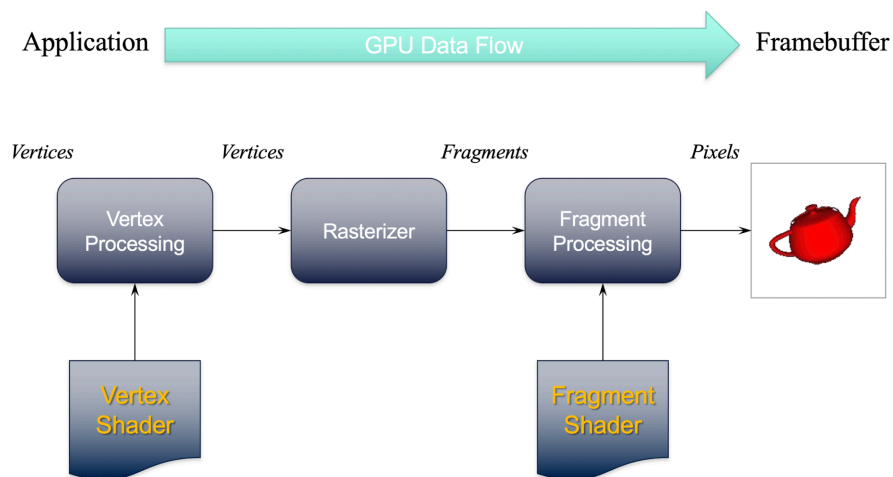


```
// Check the version
printf("OpenGL %s, GLSL %s\n",
      glGetString(GL_VERSION), glGetString(GL_SHADING_LANGUAGE_VERSION));

glutDisplayFunc(display);
glutMainLoop();

return 0;
}
```

OpenGL pipeline (Simplified)

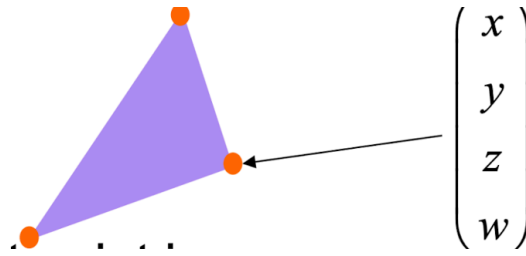


OpenGL Programming in a Nutshell

- Modern OpenGL programs essentially do the following steps
 1. Create buffer objects and load data into them
→ GPU도 동적으로 오브젝트를 생성하고 보낸다. 이를 Buffer Object라 함.
 2. Create shader programs
 3. "Connect" data locations with shader variables
 4. Render

Representing Geometric Objects

- Geometric objects are represented using vertices
- A vertex is a collection of generic attributes
 - positional coordinates
 - colors
 - texture coordinates
 - any other data associated with that point in space
- Position stored in 4 dimensional **homogeneous coordinates**



- Vertex data must be stored in vertex buffer objects(VBOs)
→ Vertex data는 VBOs에 보관
- VBOs must be stored in vertex array objects(VAOs)
→ VBOs는 VAOs에 보관
→ 대상 하나를 표현할 때 Vertex Array로 표현한다.



Homogeneous Coordinates (동차좌표계)

: n차원 사영 공간을 (n+1)개의 좌표로 나타내는 좌표계

왜 사용하는가?

: 예를 들어, 3차원을 나타내기 위해 3개의 element만 사용한다면, (x, y, z) 가 point인지 vector인지 알 수 없다. 그러나 Homogeneous Coordinate로 나타낸다면 (x, y, z, w) 의 w 를 통해 이를 표현할 수 있다. 추가로 벡터, 점의 이동 및 회전 등을 matrix 곱으로 나타낼 수 있다는 장점이 있다.

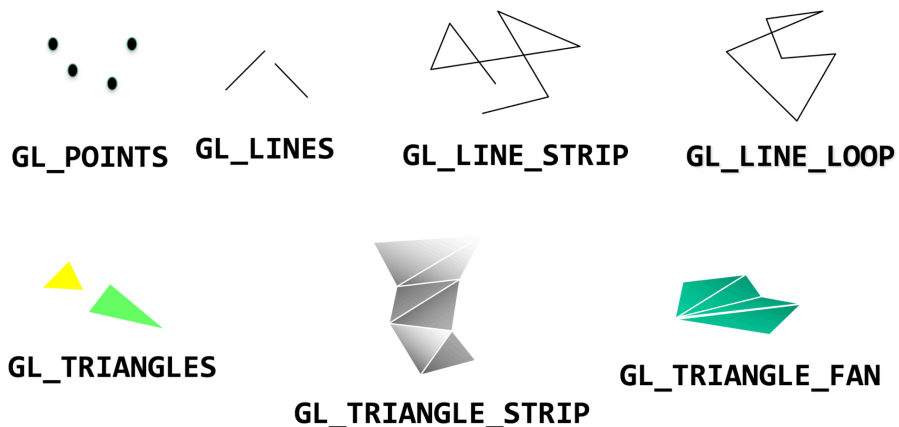
w 가 0이라면 벡터, 1이라면 점을 의미한다.

또한 동차좌표계를 사용하는 이유는 시점으로부터 방향이 중요하기 때문이다.

- OpenGL에서 표현되는 공간은 우리 눈에 보이지 않더라도 앞, 뒤의 구분이 있음
- 그렇기에 점과 벡터의 구분이 필요함
- 동차좌표가 있어 추가적으로 오브젝트의 이동을 행렬간의 곱셈으로 표현 가능
- 마지막으로 w 를 Scale Factor로 이용할 수 있다.

OpenGL's Geometric Primitives

- All primitives are specified by vertices



전체적인 과정

```
// [A] Data is Ready in CPU. Transfer the data to GPU.  
// 1. Create a Vertex Array Object  
// 2. Create a Vertex Buffer Object in the Vertex Array  
// 3. Transfer the data to the Buffer object  
  
// [B] Load your Shader Programs (이 때 Compile)  
  
// [C] Connect your data with the Shaders
```

How to send your data

1. Generate a Vertex Array

```
glGenVertexArray(...)
```

2. Bind the Vertex Array

```
glBindVertexArray(...)
```

3. Generate a Buffer Object

```
glGenBuffers(...)
```

4. Bind the Buffer Object

```
glBindBuffer(...)
```

5. Set the Buffer Object Data

```
glBufferData(...)
```

Vertex Array Objects(VAOs)

- VAOs store the data of a geometric object
 - Steps in using a VAO
 1. Generate VAO names by calling `glGenVertexArrays()`
 - Returns Handle(Unsigned Integer)
 2. Bind a specific VAO for initialization by calling `glBindVertexArray()`
 - Setting
 3. Update VBOs associated with this VAO
 4. Bind VAO for use in rendering
 - This approach allows a single function call to specify all the data for an objects.
 - Previously, you might have needed to make many calls to make all the data current.
- 하나의 함수 호출로 한 객체의 모든 데이터를 전달!

```
// Create a vertex array object  
GLuint vao;  
glGenVertexArrays(1, &vao);  
glBindVertexArray(vao);
```

Sorting Vertex Attributes

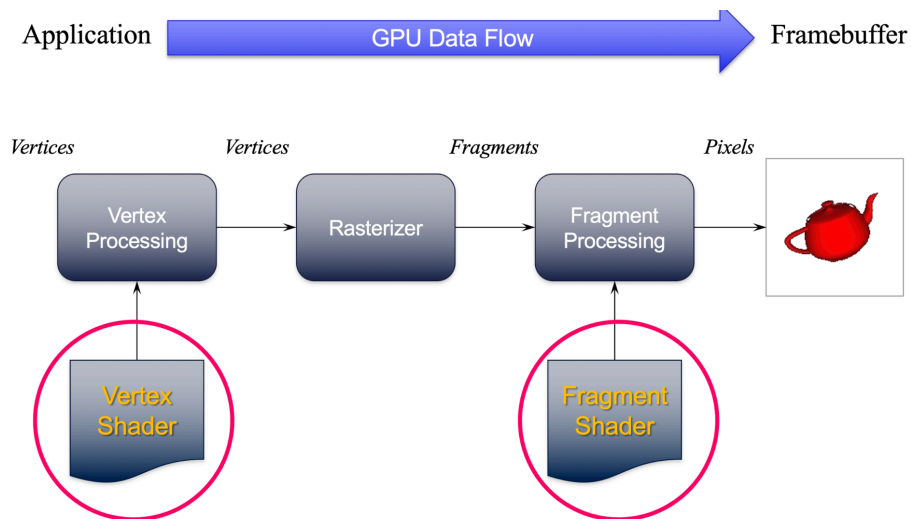
- Vertex data must be stored in a VBO, and associated with a VAO
- The code-flow is similar to configuring a VAO
 1. Generating VBO names by calling `glGenBuffers()`

2. Bind a specific VBO for initialization by calling `glBindBuffer(GL_ARRAY_BUFFER, ..)`
 3. Load Data into VBO using `glBufferData(GL_ARRAY_BUFFER, ..)`
- Bind VAO for use in rendering later `glBindVertexArray()`

```
// Create and initialize a buffer object
GLuint vbo;
glGenBuffers(1, &buffer);
glBindBuffer(GL_ARRAY_BUFFER, vbo);
glBufferData(GL_ARRAY_BUFFER, sizeof(points), points, GL_STATIC_DRAW);
```

We need shaders before drawing

이제 그릴 데이터를 Create, Initialize, Bind했으니 Shader를 작성해야 한다.



- Vertex Shader

```
#version 330

in vec4 vPosition;

void main()
{
    gl_Position = vPosition;
}
```

- Fragment Shader

```
#version 330

out fColor;

void main()
{
```

```
fColor = vec4(1.0, 0.0, 0.0, 1.0);
}
```

이제 이 Shader code 두 개를 Load 해와야 한다.

→ `#include <InitShader.h>`

```
GLuint program = InitShader("vshdaer.glsl", "fshader.glsl");
glUseProgram(program); // 등록
```

Connecting Vertex Shaders with Geometry (도형과 VShader 연결)

- **Application vertex data enters the OpenGL pipeline through the vertex shader**
- **Need to connect vertex data to shader variables**
 - Requires knowing the attribute location
 - shader 변수에 Vertex shader를 연결해야 함. 근데 누군지 알아야 하잖아.
- Attribute location can either be queried by calling `glGetVertexAttribLocation()`
 - 이 함수를 사용해서 shader 변수를 query한다.

```
// Set up vertex arrays (after shader are loaded)
int vPosition = 0;
glEnableVertexAttribArray(vPosition);
glVertexAttribPointer(vPosition, 2, GL_FLOAT, GL_FALSE, 0, BUFFER_OFFSET(0));
```

Drawing Geometric Primitives (도형 그리기)

- For contiguous groups of vertices
 - `glDrawArrays(GL_POINTS, 0, numPoints)`
- Usually invoked in display callback
 - display callback에서 주로 호출
- Initiates vertex shader
 - Vertex Shader를 실행(시작)

최종 정리

1. Creating Data (in Array form)
2. Sending the Data
 - VAO - vertex array object
 - VBO - vertex buffer object
3. Loading the shaders
4. Draw it with `glDrawArrays(...)`

A Color Cube Example and GLSL

A Color Cube Example

Rendering a Cube

- We'll render a cube with colors at each vertex
- Our example demonstrates
 - Initializing vertex data
: Vertex Data 초기화
 - Organizing data for rendering
: Rendering을 위한 데이터 생성
 - Simple object modeling
 - Building up 3D objects from geometric primitives
: Geometric primitives에서 3차원 물체 구축
 - Building geometric primitives from vertices
: 정점을 통해 Geometric primitives 생성

Initializing the Cube's Data

- We'll build each cube face from individual triangles
→ 모든 면은 삼각형을 통해 만듦
- Need to determine how much storage is required
: 필요한 vertex가 몇개인지 계산 → Array로 생성
 - (6 faces) * (2 triangles per face) * (3 vertices per triangle)
`const int numVertices = 6 * 2 * 3;`

중복되는 것이 있지만 이렇게 보내는 이유는 구조를 단순하게 하기 위함이다. (중복 허용)
- To simplify communicating with GLSL, we'll use a `vec4` class(implemented in C++) similar to GLSL's `vec4` type
→ GLSL의 `vec4` 타입과 유사한 C++ `vec4` 클래스를 사용해 GLSL과의 통신을 쉽게 함

왜 `vec3` 가 아닌 `vec4` 를 사용하느냐! → 동차좌표계를 사용하기 위함
또한 Color도 Alpha값을 포함하기 위해!

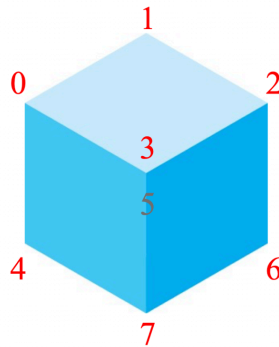
• Before we can initialize our VBO, we need to stage the data

- Our cube has two attributes per vertex
 - position
 - color
- We create two arrays to hold the VBO data

```
vec4 points[numVertices];  
vec4 colors[numVertices];
```

```
// Vertices of a unit cube centered at origin, sides aligned with axes  
vec4 vertex_pos [8] = {  
    vec4 ( -0.5, -0.5, 0.5, 1.0 ),  
    vec4 ( -0.5, 0.5, 0.5, 1.0 ),  
    vec4 ( 0.5, 0.5, 0.5, 1.0 ),  
    vec4 ( 0.5, -0.5, 0.5, 1.0 ),  
    vec4 ( -0.5, -0.5, -0.5, 1.0 ),  
    vec4 ( -0.5, 0.5, -0.5, 1.0 ),  
    vec4 ( 0.5, 0.5, -0.5, 1.0 ),  
    vec4 ( 0.5, -0.5, -0.5, 1.0 ),  
};
```

```
vec4 ( 0.5, -0.5, -0.5, 1.0 )
};
```



이때, 번호를 매길 때에는
시계 방향이나 시계 반대 방향으로 반드시 해야 한다. (일관성이 있어야 함)
→ 나중에 추가적으로 어디가 안쪽인지, 바깥쪽 면인지 알아낼 수 있음

```
// RGBA colors - Data의 순서와 동일하게 설정
vec4 vertex_colors[8] = {
    vec4 ( 0.0, 0.0, 0.0, 1.0 ), // black
    vec4 ( 1.0, 0.0, 0.0, 1.0 ), // red
    vec4 ( 1.0, 1.0, 0.0, 1.0 ), // yellow
    vec4 ( 0.0, 1.0, 0.0, 1.0 ), // green
    vec4 ( 0.0, 0.0, 1.0, 1.0 ), // blue
    vec4 ( 1.0, 0.0, 1.0, 1.0 ), // magenta
    vec4 ( 1.0, 1.0, 1.0, 1.0 ), // white
    vec4 ( 0.0, 1.0, 1.0, 1.0 ) // cyan
};
```

Generating a Cube face from vertices

```
// quad() generates two triangles for each face and assigns colors to the vertices
int Index = 0; // global variable indexing into VBO arrays
```

```
// 안에 넣는 인자는 vertex, color의 index
```

```
void quad(int a, int b, int c, int d) {
    colors[Index] = vertex_colors[a]; points[Index] = vertex_pos[a]; Index++;
    colors[Index] = vertex_colors[b]; points[Index] = vertex_pos[b]; Index++;
    colors[Index] = vertex_colors[c]; points[Index] = vertex_pos[c]; Index++;

    colors[Index] = vertex_colors[a]; points[Index] = vertex_pos[a]; Index++;
    colors[Index] = vertex_colors[c]; points[Index] = vertex_pos[c]; Index++;
    colors[Index] = vertex_colors[d]; points[Index] = vertex_pos[d]; Index++;
}
```

```
// generate 12 triangles: 36 vertices and 36 colors
```

```
void colorcube() {
    quad( 1, 0, 3, 2 );
    quad( 2, 3, 7, 6 );
    quad( 3, 0, 4, 7 );
```

```

quad( 6, 5, 1, 2 );
quad( 4, 5, 6, 7 );
quad( 5, 4, 0, 1 );
}

```

VAOs and VBOs

```

// Create a vertex array object
GLuint vao;
glGenVertexArrays(1, &vao);
glBindVertexArray(vao);

// Create and initialize a buffer object
GLuint buffer;
glGenBuffers(1, &buffer);
glBindBuffer(GL_ARRAY_BUFFER, buffer);

glBufferData(GL_ARRAY_BUFFER, sizeof(points) + sizeof(colors), NULL, GL_STATIC_DRAW);

glBufferSubData(GL_ARRAY_BUFFER, 0, sizeof(points), points);

glBufferSubData(GL_ARRAY_BUFFER, sizeof(points), sizeof(colors), colors);

// Load and use shaders
GLuint program = InitShader( "vshader.glsl", "fshader.glsl" );
glUseProgram( program );

```

Vertex Array Code

```

// set up vertex arrays (after shaders are loaded)
GLuint vPosition = glGetAttribLocation(program, "vPosition");
glEnableVertexAttribArray(vPosition);
glVertexAttribPointer(vPosition, 4, GL_FLOAT, GL_FALSE, 0, BUFFER_OFFSET(0));

GLuint vColor = glGetAttribLocation(program, "vColor");
glEnableVertexAttribArray(vColor);
glVertexAttribPointer(vColor, 4, GL_FLOAT, GL_FALSE, 0, BUFFER_OFFSET(sizeof(points)));

```

Summary

- We learnt the very basic of defining geometry and use it on GPU
- Today, we will learn how to code the shaders
- **Vertex Shader**
 - Determining the positions of vertex
: Vertex Position을 여기서 정함
 - Useful for scaling, rotating, deforming and so on
: 행렬 연산 우수
 - Also important for preparing some information sending to Fragment Shader
: Fragment Shader에 여러 가지 정보를 보내기도 함
- **Fragment Shader**

- Determining the color of each fragment
: 각각의 Fragment의 색상을 여기서 정함

OpenGL Shading Language (GLSL)

GLSL Syntax Overview

- GLSL is like C without
 - Pointers
 - Recursion
 - Dynamic memory Allocation
- GLSL is like C with
 - Built-in vector, matrix and sampler types
 - Constructors
 - A great math library
 - Input and output qualifiers

Allow us to write concise, efficient shaders.

: 위를 통해 간결하고 효율적인 셰이더 작성 가능

GLSL Syntax

- GLSL has a preprocessor

```
#version 430

#ifdef FAST_EXACT_METHOD
    FastExact();
#else
    SlowApproximate();
#endif

// Many others..
```

- All Shaders have `main()`

```
void main(void)
{
    // Code snippet
}
```

GLSL Data Types

- Scalar Types
: float, int, bool
- Vector Types
: `vec2`, `vec3`, `vec4`, `ivec2`, `ivec3`, `ivec4`, `bvec2`, `bvec3`, `bvec4`
- Matrix Types
: `mat2`, `mat3`, `mat4`

- Texture sampling

: `sampler1D` , `sampler2D` , `sampler3D` , `samplerCube`

- C++ Style Constructors

: `vec3 a = vec3(1.0, 2.0, 3.0);`

Operators

- Standard C/C++ arithmetic and logic operators
- Operator overloaded for matrix and vector operations

```
mat4 m;
vec4 a, b, c;

b = a * m; // 연산은 되는데 이 방식은 적합 X
c = m * a; // 이 방식이 적합한 방식 - 당연함 (행렬 * 벡터) -> transformation
```

Components and Swizzling

- For vectors can use `[]` , `xyzw` , `rgba` or `stpq`

```
vec4 v;
// v[1], v.y, v.g, v.t all refer to the same element

// Swizzling - 이렇게 써도 알아먹음
vec4 a, b;
a.xy = b.yx;
```

Vectors

```
// Constructors
vec3 xyz = vec3(1.0, 2.0, 3.0);
vec3 xyz = vec3(1.0); // [1.0, 1.0, 1.0]
vec3 xyz = vec3(vec2(1.0, 2.0), 3.0);

// Syntax Error
vec3 xyz(1.0, 2.0, 3.0) // ERROR!

vec4 c = vec4(0.5, 1.0, 0.8, 1.0);
vec3 rgb = c.rgb; // [0.5, 1.0, 0.8]
vec3 bgr = c.bgr; // [0.8, 1.0, 0.5]
vec3 rrr = c.rrr; // [0.5, 0.5, 0.5]

c.a = 0.5; // [0.5, 1.0, 0.8, 0.5]
c.rb = 0.0; // [0.0, 1.0, 0.0, 0.5]

float g = rgb[1] = 1.0 // indexing, not swizzling
```

Matrices

- Matrices are built-in types
 - Square

: `mat2`, `mat3`, `mat4`

- Rectangular

: `matmxn (m columns, n rows)`

→ `mat2×3`, `mat3×4`, `mat4×4`

- **Stored column major - 일반적인 행렬과 Transpose**

[참고] 좌표 변환 in 선형대수

→ 행렬에서 열이 갖는 의미도 생각해보자.

⇒ Column을 각각의 Vector로써 관찰

- Constructors

```
mat3 i = mat3(1.0); // 3*3 identity matrix

mat2 m = mat2(1.0, 2.0, // [1.0, 3.0]
              3.0, 4.0); // [2.0, 4.0] → Column Major!
```

- Accessing elements

```
float f = m[column][row]; // Column Major!

float x = m[0].x; // Treat matrix as array of column vectors

vec2 yz = m[1].yz; // Can swizzle too!
```

Vectors and Matrices

- Vector and Matrix operations are easy and fast

```
vec3 xyz = // ...

vec3 v0 = 2.0 * xyz; // Scale
vec3 v1 = v0 + xyz; // Component-wise
vec3 v2 = v0 * xyz; // Component-wise
// 원래 벡터에는 곱하기 연산이 없지만, 구현적인 측면에서 편리함을 위해 구현된 연산임

mat3 m = // ...
mat3 t = // ...

mat3 mt = m * t; // matrix * matrix
vec3 xyz2 = mt * xyz; // matrix * vector
vec3 xyz3 = xyz * mt; // vector * matrix (= transposed_matrix * vector)
```

Qualifiers

- in, out
 - Copy vertex attributes and other variables to/from shader

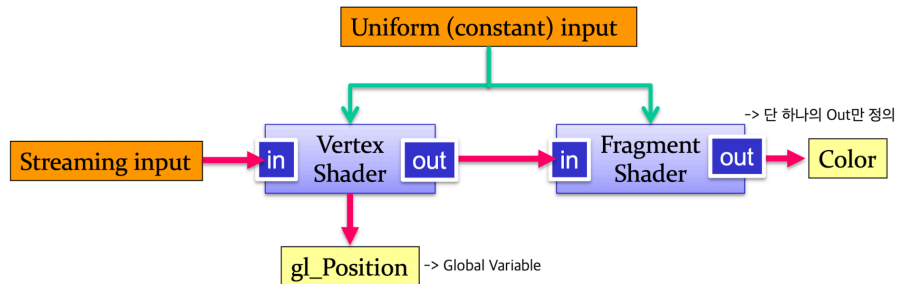
`in vec2 tex_coord;`

`out vec4 color;`

- Uniform : variable from application

```
uniform float time;
uniform vec4 rotation;
```

그리기 전에 Setting하는 것 → GPU는 이를 상수로 취급
→ cpp에서 상수로 전달함 (시간같은 걸 많이 전달함)



중요!

- Vertex shader는 gl_Position를 반드시 설정 (Global Variable)
- Fragment Shader는 반드시 Color 값을 내보내야 함(out 단 한가지 반드시)

Flow Control

```
if
if-else
[expression] ? [true-expression] : [false-expression]
while, do-while
for
```

Functions

- Built-in
 - Arithmetic
 - sqrt, power, abs
 - Trigonometric
 - sin, asin
 - Graphical
 - length, reflect
- User Defined

Built-in Variables

- gl_Position
 - : output position from vertex shader
- gl_FragColor
 - : output color from fragment shader → old version에서만 사용

Simple Shaders for Cube

```
// Vertex Shaders
#version 330

in vec4 vPosition;
in vec4 vColor;
out vec4 color;

void main()
{
    color = vColor;
    gl_Position = vPosition;
}
```

```
// Fragment Shader
#version 330

in vec4 color;
out vec4 FragColor;

void main()
{
    FragColor = color;
}
```

InitShader.h → 생략

Determining Locations After Linking

- Assumes you already know the variables' name

```
GLuint name1 = glGetAttribLocation(program, "name1");

GLuint name2 = glGetUniformLocation(program, "name2");

// 실제 구현
// Shaders가 로드되고, Vertex Arrays가 Set up된 후
GLuint vPosition = glGetAttribLocation(program, "vPosition");
glEnableVertexAttribArray(vPosition);
glVertexAttribPointer(vPosition, 4, GL_FLOAT, GL_FALSE, 0, BUFFER_OFFSET(0));

GLuint vColor = glGetAttribLocation(program, "vColor");
glEnableVertexAttribArray(vColor);
glVertexAttribPointer(vColor, 4, GL_FLOAT, GL_FALSE, 0, BUFFER_OFFSET(sizeof(points)));
```

Initializing Uniform Variable Values

- Uniform Variables

```
glUniform1f(idx, value);
glUniform4f(idx, x, y, z, w);
```

```

GLfloat transpose = GL_TRUE;
GLfloat mat[3][4][4] = {...}; // (), row, column (in C++)

glUniformMatrix4fv(idx, 3, transpose, mat);

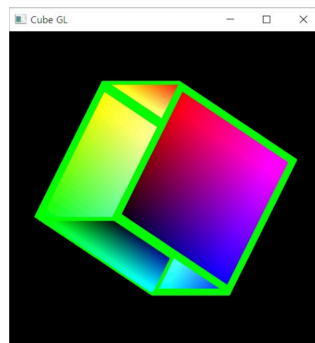
// idx : GLuint

```

Shader Applications

- **Moving vertices**
 - Morphing
 - Wave motion
 - Fractals
- **Lighting**
 - More realistic models
 - Cartoon shaders
- And more and more!

One little problem - Incorrect Depth Handling



→ 보면 앞면, 뒷면이 제대로 구분이 안된 것 같이 보임. 이를 제대로 나오게 하려면?

원래라면, 삼각형의 모든 정보를 가지고 Sorting해야 했다.

→ 근데 이는 너무 비효율적이라 다른 방법을 생각함

→ 바로 Z-buffer를 사용하기로 함!

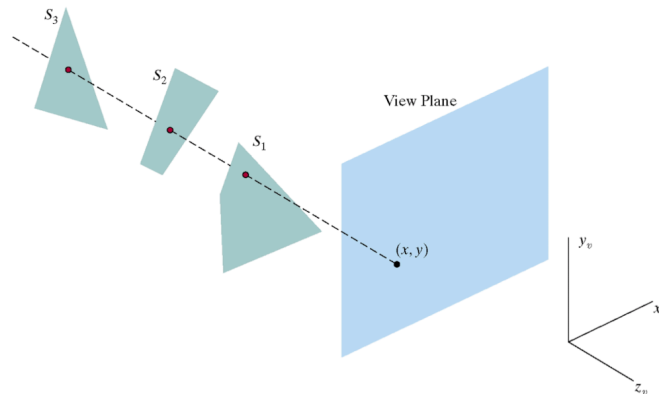
Depth-Buffer (Z-Buffer)

- Z-Buffer has memory corresponding to each pixel **location for storing the current depth value. (distance from view plane)**
 - Useful for determining whether a new drawing pixel is visible or not
 - Is **visible** when the its distance is closer than the previously stored one
 - Is **not visible** when it is farer
- 위치마다 컬러값을 저장하는 것 : Color Buffer(Screen Buffer)
- 더 추가적인 정보를 담아보자 : Z-Buffer(Depth Buffer)
 - Z를 보고 누구를 위에 그릴지 판단한다. (그리는 순서와 관계 없이)

→ 누가 더 가까운지를 판단하는 것이 중요하다.

- 언제보일 거임?

→ 이전에 칠했던 것보다 가까우면 보임. 멀면 안 그릴 거임.



Enabling the Depth Buffer in OpenGL

OpenGL에서는 Z-Buffer를 사용할 수 있음. 그러나 Default가 아니기에 설정해야 함.

→ 이는 Shader가 하는 작업이 아니라는 것을 명심.

```
// On Initialization
glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGBA | GLUT_DEPTH);

// On Drawing
glClearColor(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
glEnable(GL_DEPTH_TEST);

// Double Buffering을 사용하기 위해
glFlush();
glutSwapBuffer(); // glFlush(); 뒤에 반드시 호출
```



Double Buffering

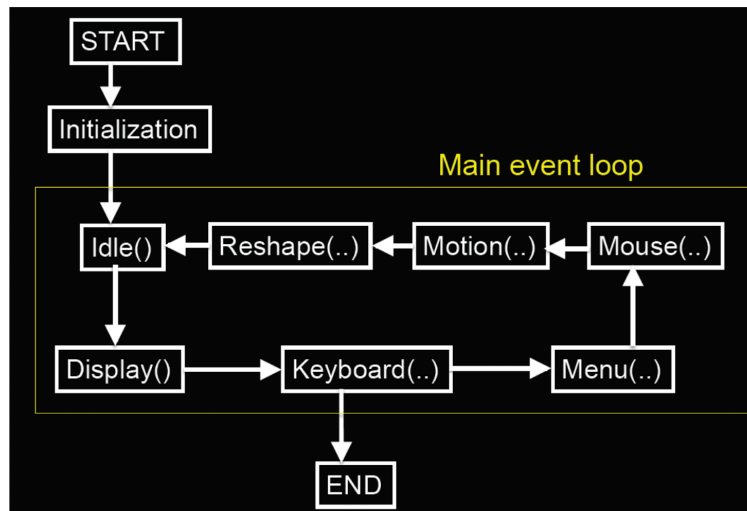
프론트 버퍼는 현재 모니터에 표시되고 있는 이미지이고, 백 버퍼는 그래픽 카드(GPU)가 다음 프레임을 그리는 동안 사용되는 작업 공간입니다. 백 버퍼에 새로운 이미지가 완성되면 프론트 버퍼와 빠르게 '스왑(swap, 바꿔치기)'하여, 완성된 화면만을 보여줌으로써 화면 깜빡임 없이 부드러운 화면 전환을 가능하게 하는 것이 더블 버퍼링 기술의 핵심입니다.

→ 버퍼를 두 개 두어서 깜빡임 현상을 없애려고 함 (그림이 복잡할 때 사용)

→ 그래서 코드에서도 DOUBLE Buffer를 사용하려면, flush 이후에 `glutSwapBuffer();` 를 사용해주어야 한다. (프론트 버퍼와 백 버퍼 교체)

Interaction : Callbacks

GLUT Program with Callbacks



→ display도 사실 Callback 함수임

Event Types

- Window
 - : resize, expose, iconfiy
- Mouse
 - : click one or more buttons
- Motion
 - : move mouse
- Keyboard
 - : press or release a key
- Idle
 - : non-event
 - Define what should be done if no other event is in queue.

GLUT Callbacks

GLUT recognizes a subset of the events recognized by any particular window system(Windows, X, Macintosh)

- `glutDisplayFunc`
- `glutMouseFunc`
- `glutReshapeFunc`
- `glutKeyboardFunc`
- `glutIdleFunc`
- `glutMotionFunc` , `glutPassiveMotionFunc`

Types of Callbacks

- `Display()`
 - : when window must be drawn
- `Idle()`
 - : when no other events to be handled

- `Keyboard(unsigned char key, int x, int y)`
: key pressed
- `Menu(...)`
: after selection from menu
- `Mouse(int button, int state, int x, int y)`
: mouse button
- `Motion(...)`
: mouse movement
- `Reshape(int w, int h)`
: window resize
- **Any callback can be NULL**

Example : Idling Callback

- Idling Callback is useful for defining a periodic work.
E.g., Making an animation (rotating a cube and so on)
- How to use
 1. Registering your function as an Idling Callback function

```
glutIdleFunc(myIdle);
```

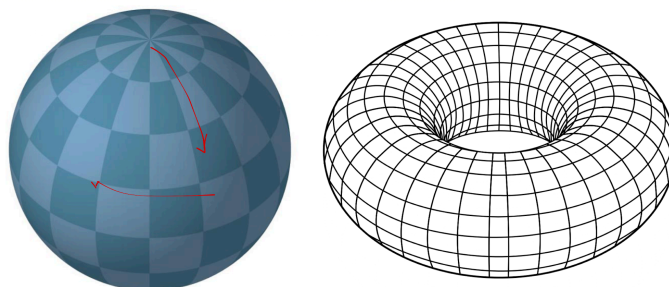
2. Implement what you want to do repeatedly

```
void myIdle()
{
    // do something like periodic job
    Sleep(16);      // wait for 16 ms
    glutPostRedisplay(); // invoke redrawing
}
```

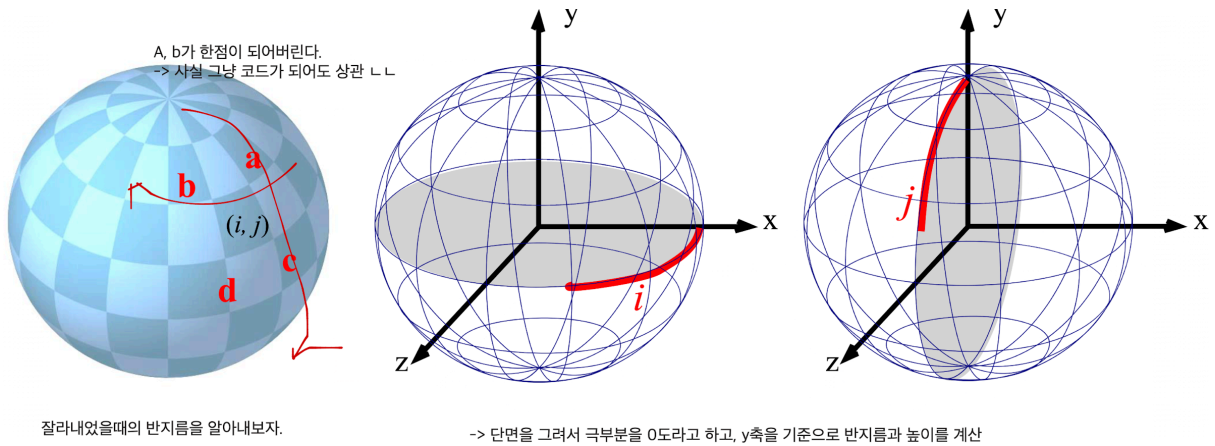
Other Geometric Objects

Coding Practice : Sphere and Torus

원과 관련된 것들 → 높이를 나누기보다는, 각도를 쪼개서 각도를 조절하는 것이 더 좋은 아이디어!



- 구를 생각할 때는 3차원을 2차원 곡면에 있는 것처럼 해보자.
즉, (x, y, z) 로 생각하지 말고 (i, j) 로 생각해라 (각도에 대한 좌표 - 구면좌표계)



여기까지 정리!

1. Geometry를 표현할 수 있어야 한다. - 현재 여기까지 완료!
2. 카메라를 두어야 한다.
3. 재질과 조명 세팅을 한다.

Geometric objects and Its representations

Scalars

- Scalar는 Scalar Field에서 정의된다.
- 가감승제, 0, 1, 제곱 연산이 모두 가능하다.

Vectors

Vector 자체도 각 지점에서 원점을 바라본 것이나 다름 없다고 생각해보자.

- Vector는 vector space에서 정의된다.
- 덧셈 및 뺄셈이 가능하다.
- Zero vector : 0
- Scalar Multiplication도 가능하다.

[중요]

벡터를 더하는 것은 가고 또 가면되는 느낌이지만,

빼는 것은 뒤의 벡터에서 앞의 벡터를 바라보는 느낌이다.

→ 빼는 것의 의미가 더 크다.

그러나, $r = a * P + b * Q$ 는 어쩌면 물리적인 의미를 가질 수 있다. (원점과 관계없는 의미)

1. $a + b = 1$ 일 때, r 은 두 벡터를 이은 직선 위의 어떤 점(Interpolation 혹은 Extrapolation)

2. $a + b = 0$ 일 때, $d = Q - P$ 라면

$$R = P + \alpha * d$$

$$\text{즉, } R = P + \alpha * (Q - P) = (1 - \alpha) * P + \alpha * Q$$

Lines and line Segments

- Parametric(매개변수) form of line

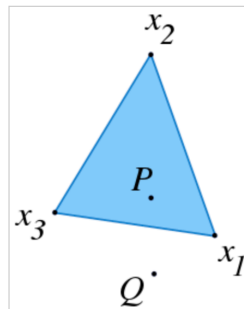
$$P(\alpha) = P_0 + \alpha * d$$

→ P_0 에서 시작해서 d 방향으로 α 만큼 간다.

- Line segment between Q and R

$$P(\alpha) = (1 - \alpha) * Q + \alpha * R \quad \text{for } 0 \leq \alpha \leq 1$$

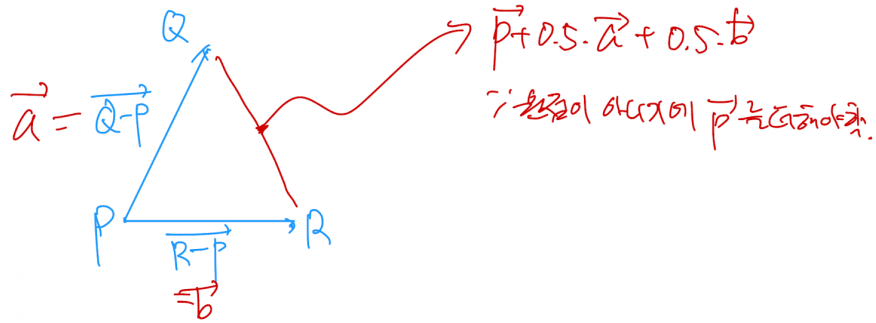
Convex Combination : 계수의 합계가 1이 되는 경우



$$p = a_1x_1 + a_2x_2 + \dots + a_nx_n$$
$$\sum a_n = 1, a_n \geq 0$$

→ 어떤 도형이던 관계 없다. 주어진 점들로 생성된 도형 안의 점을 나타낼 수 있음.

<삼각형 내부의 점>



↳ 삼각형 내부.

$\vec{p} + (1-\alpha) \cdot \vec{a} + \alpha \cdot \vec{b}$ ($\alpha < 1$) 이면,
삼각형 내부의 점을 구할 수 있음.

⇒ 만일 α 에 대한 조건을 해제하면,
결론적으로 줄, 하나의 값을 가지고 하나의 평면을
표현할 수 있다는 것을 알게 됨

→ 두 벡터가 뻗어나오는 지점이 원점이 아니라면 반드시 그 벡터를 더해줘야 한다.

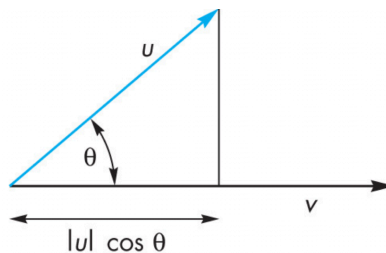
결론적으로, 만일 $\alpha = 1$ 이면 직선 위의 것이고, α 가 0 ~ 1사이라면 삼각형 내부일 것이다.

그런데, α 에 대한 조건을 해제한다면 (모든 실수라면) 평면 전체를 정의할 수 있다.

즉, 하나의 점, 두 개의 벡터를 가지고 하나의 평면을 표현할 수 있게 된다. (α 가 모든 지점을 정의함)

Dot Product (Projection)

- Dot product는 한 벡터에서 다른 벡터로의 projection을 표현한다.
- $u \cdot v = u_1v_1 + u_2v_2 + u_3v_3 = |u||v|\cos(\theta)$
- $projection_v(u) = (u \cdot v)v/|v|^2$



값보다는 부호가 더 의미를 갖는다.

- $+$: 두 벡터가 같은 방향일수록 커짐
- 0 : 두 벡터는 서로 직교
- $-$: 두 벡터는 서로 다른 방향일수록 작아짐

→ u, v 의 크기에 따라 달라질 수 있기에 경향성만 파악하고, **0일 때 직교**라는 것에 주의할 것!

벡터에서는 서로 관계가 없다고 할 때, 직교한다는 표현을 많이 사용한다.

Perpendicular == Orthogonal

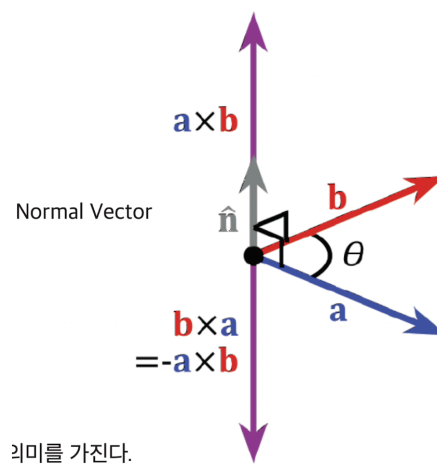
Cross Product

$$\begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} \times \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} = \begin{pmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{pmatrix}$$

- Cross product는 a 와 b 의 직교하는 vector를 생성한다.
- Right-hand rule을 사용 ($A \times B$ 일 때 A 는 B 가 가까운 방향으로 돌아가는 방식이다.)
 - 첫번째 벡터가 검지이다. 엄지는 생성된 벡터의 방향

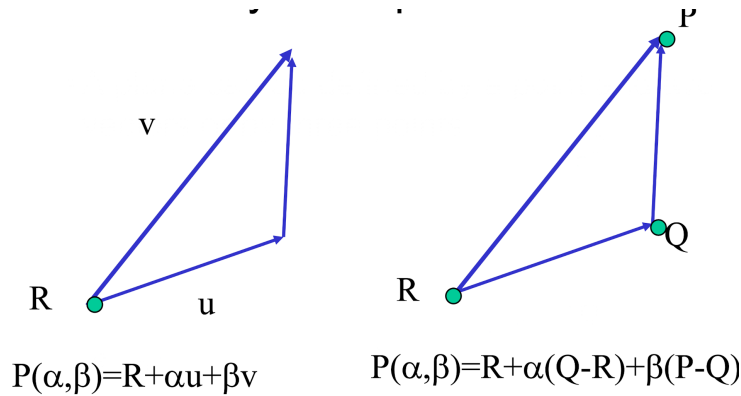
Cross Product는 기본적으로 두 가지의 의미를 갖는다.

- 벡터의 길이 : 직교할수록 커짐 (얼마나 많이 벌어졌는가 - 같은 벡터라면 0임.)
 - 물론 a, b 의 크기에 따라 달라지기에 절대적인 것은 X (**경향성만 파악 후 실제로는 0일 때 더 큰 의미**)
- 벡터의 방향 : 직교하는 벡터가 생성됨



Planes

- Plane은 한 점과 2개의 벡터로 정의됨. 혹은 세 개의 점으로 정의됨



→ 일전에 본 삼각형 위의 점의 확장버전이다. 여기서는 alpha와 beta의 값이 모든 실수이다.

Planes and normal

- 평면은 한 점과 두 개의 벡터로 정의된다. (반복)
- 이 때, 두 개의 벡터는 평행하면 안 된다.
- 매개변수로 표현해본다면

$$T(\alpha, \beta) = P_0 + \alpha * u + \beta * v \quad (\alpha, \beta \text{ are Scalar})$$
- $n = u \times v / |u \times v|$ 가 Normal Vector
- $n * (P - P_0) = 0$ 이면, P가 plane 위의 점이라는 것!
 → 내적이 0이라는 것은 n과 $P - P_0$ 이 직교한다는 것!

평면의 정의는 한 점과 두 벡터 혹은 한 점과 그 두 벡터의 수직인 벡터로 정의될 수 있다.

→ 예를 들어 한 점과 그 평면에 수직인 벡터(평면 방향)이 결정되면 하나의 평면이 정의된다.

Geometric Transformations

Transformations

- **Linear Transformations**
 : 벡터의 각 성분을 조작하여 (일정한 비율(스칼라)을 곱하기, 다른 성분과 더하기 등)으로 만들어내는 것을 말함.
 - 확대, 축소, 회전, 쉬어링, 대칭 가능
 - 치명적인 문제 : 이동이 안 됨 (그래서 Homogeneous Coordinate를 사용 → Affine)
- **Rigid Transformations**
 : Linear Transformations의 종류 중 하나. 강체 변형이라 한다. 임의의 두 점이 같이 이동했을 때, 그 두 점 사이의 길이가 동일할 때를 말한다. 즉, 몇 개의 점이던 어떤 도형이 이동 회전 등을 할 때도 동일한 모양을 유지하는 것을 말한다.
 - Affine Transformation 중 에 Rotation과 이동만 가능하면, 강체 변형이라 한다.
- **Affine Transformations**
 - Homogeneous Coordinate를 사용하는 변경
 - Linear Transformation + 이동
- **Projective Transformations**
 - 행렬에서 마지막 Row가 (0,0,1)이 아닌 Affine transformation
 - 투영 변형 혹은 Perspective transformation이라 한다.

Homogeneous Coordinates

- 3차원에서 모든 Affine Transformation은 4 * 4 행렬로 표현된다.

$$T(\mathbf{p}) = \begin{pmatrix} \mathbf{M}_{3 \times 3} & \mathbf{T}_{3 \times 1} \\ 0 & 1 \end{pmatrix} \begin{pmatrix} \mathbf{p}_{3 \times 1} \\ 1 \end{pmatrix}$$

- 기존 3 * 3 행렬로는 이동이 표현이 안 된다.
- 표현이 부족하다면, 재료를 늘리면 된다는 발상

행렬에서 Column은 가지고 있는 재료의 갯수이다.

→ Row는 성분 비율을 나타냄

- 동차 좌표계에서는 마지막 Row를 0, 0, 0, 1로 고정
- 나머지 12개의 원소 값을 변경해서 활용한다.

Projective Spaces

- Homogeneous Coordinates
 - $(x, y, z, w) = (x/w, y/w, z/w, 1)$ 가 가능함 (비율로 보기에!!)
 - 값은 4개지만, 3차원 좌표이다.
 - w는 x, y, z를 w배만큼 뺀다는 느낌.
 - 원래 기본적인 행렬은 덧셈만 가능했음 (성분끼리 더하기 혹은 상수배만 됨)
근데 Homogeneous Coordinate를 통해 나누기가 가능해짐
 - Perspective projection을 Handling하기에 유용하다.
 - 마지막 row가 0, 0, 0, 1이 아니라면?

$$\begin{pmatrix} 1 \\ 0 \\ x \end{pmatrix} = \begin{pmatrix} \square & \square \\ 1 & 0 & 0 & 0 \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

→ $\frac{x}{x}, \frac{y}{x}, \frac{z}{x}$
→ 이걸 보자!

→ 즉, w가 아니라 x로 나눠짐.

- 하지만, 동일한 의미를 갖는다고 Algebraically(대수적으로)에서 동일한 것은 아님!

$$(1,0,0,1) + (1,1,0,1) = (2,1,0,2) = (1, \frac{1}{2}, 0, 1)$$

||

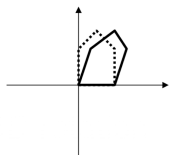
||

≠

$$(1,0,0,1) + (2,2,0,2) = (3,2,0,3) = (1, \frac{2}{3}, 0, 1)$$

Affine Transformation의 예시

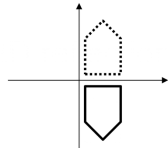
- 2D Shear



$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & d & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x + dy \\ y \\ 1 \end{pmatrix}$$

- 2D Reflection

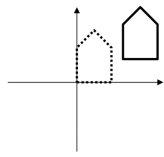
• 2D reflection



$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ -y \\ 1 \end{pmatrix}$$

- 2D Translation

• 2D translation



$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ 1 \end{pmatrix}$$

• 2D Pivot-Point Rotation

- Pivot Point (x, y) 를 기준으로 Rotation을 하는 것이다.

$$\rightarrow T(x, y) * R(\theta) * T(-x, -y)$$

⇒ 원점으로 이동 → 회전 → 원래 자리로 이동

$$\begin{aligned} &= \begin{pmatrix} 1 & 0 & x \\ 0 & 1 & y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & -x \\ 0 & 1 & -y \\ 0 & 0 & 1 \end{pmatrix} \\ &= \begin{pmatrix} \cos \theta & -\sin \theta & x(1 - \cos \theta) + y \sin \theta \\ \sin \theta & \cos \theta & y(1 - \cos \theta) - x \sin \theta \\ 0 & 0 & 1 \end{pmatrix} \end{aligned}$$

• 2D Fixed-point Scaling

- Fixed-point (x, y) 를 기준으로 크기를 키우는 것

$$\rightarrow T(x, y) * S(s_x, s_y) * T(-x, -y)$$

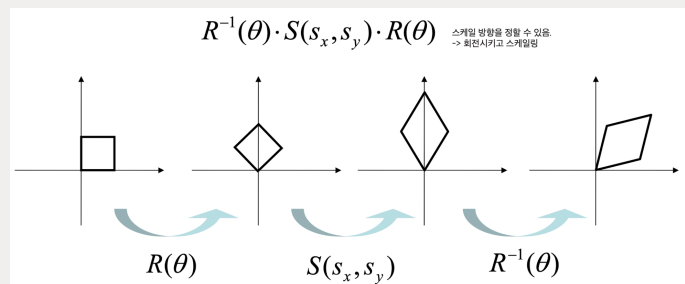
⇒ Scaling도 Rotation과 동일하다.

$$\begin{aligned} &= \begin{pmatrix} 1 & 0 & x \\ 0 & 1 & y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & -x \\ 0 & 1 & -y \\ 0 & 0 & 1 \end{pmatrix} \\ &= \begin{pmatrix} s_x & 0 & (1 - s_x) \cdot x \\ 0 & s_y & (1 - s_y) \cdot y \\ 0 & 0 & 1 \end{pmatrix} \end{aligned}$$



[Scaling Direction]

- 스케일 방향을 결정할 수 있다. (회전을 이용함)



- 3D Rotation in Matrix

• 3D rotation

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

→ 축을 바라보는 방향에서 시계 반대 방향으로 회전함 (+)

→ y축으로 회전하게 되면 좁은쪽으로 회전하면 (z→x) : 즉 각도가 마이너스임(그래서 sin의 부호가 바뀜)

- Rotation에서 중요한 것
 - 길이 변화 없다.
 - 각도 변화 없다.
- 기본적으로 축을 생각한다.
 - 좌우
 - 높이
 - 깊이

3D Rotation representation

3차원 회전은 기본적으로 축이 필요하다.

- 자유도 : 어떤 상태를 표현할 변수의 갯수

2차원 좌표계를 기준으로 생각해보자.

어떤 회전을 표현할 때는 각도 하나로 표현한다.

혹은 어떤 행렬로 표현한다.

결론적으로, 어떤 현상을 표현할 때 얼마나 많은 숫자(자유도)가 필요한지는 계산 전에 확인할 수 있다.

→ 근데 그걸 실제로 구현할 때는 더 많은 숫자가 들어가는 경우가 많다.

- 3차원 회전을 표현할 때 다양한 방법들이 있다.
 - **Rotation Matrix**
 - 결과적으로 얻어지는 것
 - **Euler angle**
 - 어떤 회전이던 간에 x, y, z 축의 회전의 융합으로 표현할 수 있다.
 - 제일 직관적임
 - **Rotation Vector**
 - 가장 Compact하다.
 - 벡터의 크기는 각도에 해당함
 - 방향의 회전축을 나타냄

- **Axis-Angle**
 - 사실상 Rotation Vector와 다를게 없음
 - 방향 벡터는 $(x, y, \sqrt{x^2 + y^2})$ 로 표현
- **Unit Quaternion**

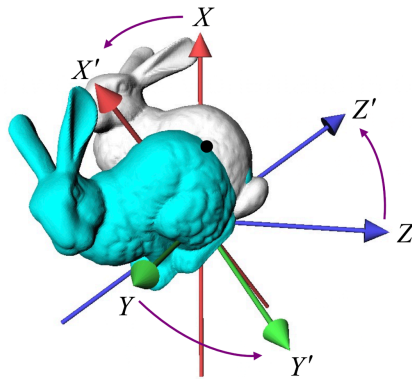
→ 하나만 사용하는게 아니라 이것 저것 섞어서 쓴다. 결론적으로는 Rotation Matrix 이용

z축을 향하여 가는 비행기를 생각했을 때,

- 위 아래 : pitch (x축 회전)
- 좌 우측 : yaw (y축 회전)
- 그냥 돌기 : rolling (z축 회전)

Rotation in 3D

한 Rigid object에 임의의 두 방향이 주어진다. 한 방향에서 다른 방향으로 회전시키려면 얼마나 많은 회전이 필요할까?

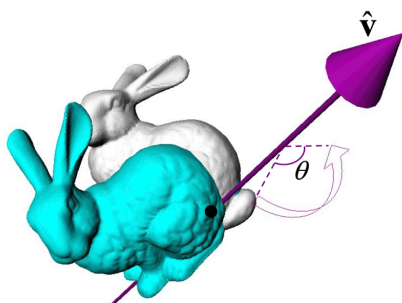


Euler's Rotation Theorem

The general displacement of rigid body with one point fixed is "a rotation" about some axis.

- 다른 말로,
 - 임의의 3차원 회전은 회전에 대한 회전으로 바뀔 수 있음
 - 모든 3차원 회전은 바뀌지 않는 한 벡터를 남긴다.

Rotation Vectro and Axis-Angle



$\hat{v} = (\hat{x}, \hat{y}, \hat{z})$: unit vector
 θ : scalar angle

- Rotation vector (3 parameters)

$$\mathbf{v} = \theta \hat{\mathbf{v}} = \theta(\hat{x}, \hat{y}, \hat{z})$$

- Axis-Angle (4 parameters)

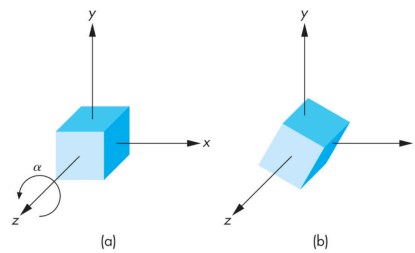
$$\mathbf{v} = (\theta, \hat{x}, \hat{y}, \hat{z})$$

→ 크기는 1임

3D Rotation Matrix

- Z-axis

$$\mathbf{R} = \mathbf{R}_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



- X-axis

$$\mathbf{R} = \mathbf{R}_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Y-axis

$$\mathbf{R} = \mathbf{R}_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3D Rotation About Arbitrary Axis

- u 벡터 기준으로 회전하고 싶다! (u : 주어진 Given Rotation Axis)

→ Rotate about x and y axes to make u align with the z -axis

→ 알고 있는 축으로 옮기고 회전 → 다시 원래 축으로 돌려 놓기

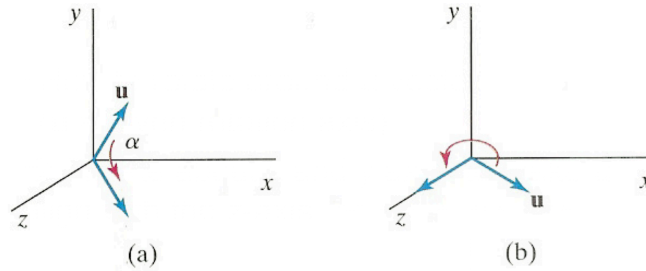


FIGURE 5-45 Unit vector u is rotated about the x axis to bring it into the xz plane (a), then it is rotated around the y axis to align it with the z axis (b).

먼저, u 를 xz 평면에 갖다 붙일 생각임 그러면 각도를 알아야겠지?

→ u 를 yz 평면에 정사영 시키고 $\cos A$, $\sin A$ 를 구한다. (각각 내적, 외적으로 구하기)

- Rotate u onto the z -axis
 - u' : u 를 yz -plane에 Projection하고, angle a 를 계산한다.
 - u'' : u 를 x -axis에 대해 angle a 만큼 회전

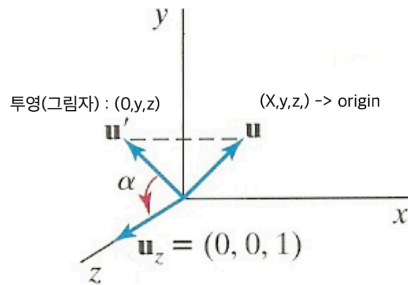


FIGURE 5-46 Rotation of u around the x axis into the xz plane is accomplished by rotating u' (the projection of u in the yz plane) through angle α onto the z axis.

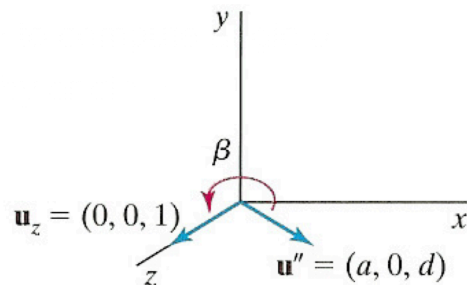


FIGURE 5-47 Rotation of unit vector u'' (vector u after rotation into the xz plane) about the y axis. Positive rotation angle β aligns u'' with vector u_z .

- u' 를 x -axis에 대해 Rotate 시켜 z -axis위에 올라가게 하기
 - $u = (a, b, c)$ 라 하고, $u' = (0, b, c) \rightarrow yz$ 평면에 투영
 - $u_z = (0, 0, 1)$ 이라 하자

- angle a 는 다음과 같이 계산됨

$$\cos \alpha = \frac{\mathbf{u}' \cdot \mathbf{u}_z}{\|\mathbf{u}'\| \|\mathbf{u}_z\|} = \frac{c}{\sqrt{b^2 + c^2}}$$

$$\begin{aligned} \mathbf{u}' \times \mathbf{u}_z &= \mathbf{u}_x \|\mathbf{u}'\| \|\mathbf{u}_z\| \sin \alpha \Rightarrow \sin \alpha = \frac{b}{\|\mathbf{u}'\| \|\mathbf{u}_z\|} = \frac{b}{\sqrt{b^2 + c^2}} \\ &= \mathbf{u}_x \cdot b \end{aligned}$$

- 위에서 구한 $\cos(\alpha)$ 와 $\sin(\alpha)$ 를 이용해 Rotation Matrix를 만들 수 있다.

$$\mathbf{R}_x(\alpha) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \frac{c}{\sqrt{b^2 + c^2}} & \frac{-b}{\sqrt{b^2 + c^2}} & 0 \\ 0 & \frac{b}{\sqrt{b^2 + c^2}} & \frac{c}{\sqrt{b^2 + c^2}} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad \begin{array}{l} \rightarrow \text{각도를 -로 하면 원래대로 복원됨} \\ \rightarrow \cos, \sin \text{을 바탕으로 사용} \end{array}$$

결론 : $x \rightarrow y \rightarrow z \rightarrow -y \rightarrow -x$ 순으로 회전 (근데 이걸 되게 복잡한 방법이긴 하다.)

[3D Rotation] 임의의 축으로 회전하기

임의의 축으로 회전하기 3D 환경에서 x축, y축, z축으로 회전하는 건 4x4 matrix로 할 수 있다. 그러면 임의의 축으로 회전하는 건 어떻게 할까? 순서 1. 임의의 축이 좌표축과 평행할 때 (Special Case) 회전 2. 좌표축과 평행하지 않은 임의의 축 회전 - 5 Step이 있음 - α 와 β 를 구하는 법 - 다른 형태의 matrix 임의의 축이 좌표축과 평행할 때 (Special

<https://inyongs.tistory.com/133>

Graphics

- 당연히 angle α 를 구할 수도 있다.

$$\text{atan2}\left(\frac{c}{\sqrt{b^2 + c^2}}, \frac{b}{\sqrt{b^2 + c^2}}\right) \rightarrow \text{이건 나중에}$$

Gimble

위는 임의의 축에 대한 것이었고, 실제로는 x, y, z 축으로 얼마나 돌릴지 알고 있을거임!

그래서 Euler Angle을 더 많이 사용한다. (직관적임)

- Gimble은 Aircraft나 Camera(삼각대)에 많이 사용된다.

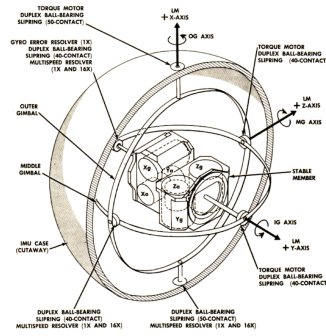


Figure 2.1-24. IMU Gimbal Assembly



Euler Angles

- 세 개의 Orthogonal Axes를 활용해 회전
- 12개의 조합이 존재한다.

- XYZ, XYX, XZY, XZX
- YZX, YZY, YXZ, YXY
- ZXY, ZXZ, ZYX, ZYZ

- Gimble Lock
 - 그러나 Gimble에는 치명적인 문제가 있음
 - 세 번을 회전한다고 했을 때, 가운데 축이 90도가 되면 첫 번째 그리고 세 번째의 축 회전이 동일하게 적용되어 버린다.
 - 근데 딱 90도에 이상해지지는 게 아니라 그 주변 값부터 이상해진다. (Singular : 이상 값)
- 또 다른 문제가 있다. 각도에 대해서 값을 사용하기 어렵다는 것이다.
 - 예를 들어 각도의 평균을 구할 수 있을까? → X
 - 가깝다는 걸 각도 자체로 표현할 수 없다.
 - 이는 선형이 아님에서 생기는 문제이다. (폐곡선을 막혀있음)
 - 또한 이러한 이유로 인해 최단거리를 갈 수 없다. (Lerping에도 문제가 있음)

음.. 그럼 다른 애를 쓰면 되겠네?

A : 응 안돼

Arbitrary rotation can be represented by three rotation along x,y,z axis

$$R_{XYZ}(\gamma, \beta, \alpha) = R_z(\alpha)R_y(\beta)R_x(\gamma)$$

$$= \begin{bmatrix} C\alpha C\beta & C\alpha S\beta S\gamma - S\alpha C\gamma & C\alpha S\beta C\gamma + S\alpha S\gamma & 0 \\ S\alpha C\beta & S\alpha S\beta S\gamma + C\alpha C\gamma & S\alpha S\beta C\gamma - C\alpha S\gamma & 0 \\ -S\beta & C\beta S\gamma & C\beta C\gamma & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

→ 행렬끼리의 덧셈은 의미가 달라진다. (곱셈은 그 의미를 유지)

→ 즉, 행렬끼리도 평균을 구할 수 없다는 것이다. (회전에서는 그 반지름이 달라지게 된다.)

- Euler Angle은 그 의미가 너무 Ambiguous하다.
 - 서로 다른 Euler Angle은 같은 방향을 의미할 수 있다.

$$R_1 = (r_x, r_y, r_z) = (\theta, \frac{\pi}{2}, 0) \quad \text{and} \quad R_2 = (0, \frac{\pi}{2}, -\theta)$$

- 이 Ambiguity는 Interpolation으로 만들어진 프레임을 사용하는 Animation에서 예상치 못한 결과를 만들어낸다.

→ **Rotation의 중간을 찾는건 아주 어렵다.**

Smooth Rotation

| 그럼 부드럽게 하려면 어떻게 해야할까? (Interpolation 어케 씬?)

Create transformations from M_0 to M_n **smoothly**

- Problem: find a sequence of model-view matrices $M_0, M_1, \dots, M_n$ for each frame to see a smooth transition

| Euler angles를 활용한 한 가지 방법이 있다.

- Find $R_0 = R_{0z} R_{0y} R_{0x}$ and $R_n = R_{nz} R_{ny} R_{nx}$
- Then, Create a sequence of rotation $R_0, R_1, \dots, R_n$:
 $R_i = R_{iz} R_{iy} R_{ix}$ (where, i_x, i_y, i_z is the interpolated angles from the beginning and the end)
- Not very effective!
- Quaternions can do it better!

27

→ 일반식을 구해서 사용하는 형태

→ 효율적이지 않다. 당연하지 최단 거리는 안되니깐 $\pi\pi\pi$ (Quaternions이 더 낫다)

Quaternions

| 상식으로만 알고 있자.

- 복소수를 이용한 방법
- 결론적으로 회전을 복소수로 표현하는 것이다.

Requires one real and three imaginary components **i, j, k**

$$q = q_0 + q_1\mathbf{i} + q_2\mathbf{j} + q_3\mathbf{k} \quad \rightarrow \text{실수부 하나와, 허수부 3개로 이루어짐 (4차원)}$$

→ 평균 계산 가능 + Interpolation 생성 가능
→ Slerp - Interpolation

<중간 범위 끝>